FrontierAtlas / GraphOne AI Intelligence Pipeline: Architectural Blueprint

System: FrontierAtlas / GraphOne AI Data Intelligence Engine
SSoT Reference: `GEMINI.md`, `GUARDRAILS.md`, `CODING_STANDARDS.md` | **Risk Tier:** Commercial / Production
Runtime: Python 3.11+ Async Native (`asyncio`, `httpx`, `curl-cffi`, `openpyxl`, `networkx`, `pydantic v2`)

1. System Architecture & Uni-Directional Pipeline Flow

The pipeline executes a strict uni-directional flow from discovery through resilient extraction to multi-target export:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                          FRONTIERATLAS ARCHITECTURE                          │
└──────────────────────────────────────────────────────────────────────────────┘

[ Discovery / Sources ]
      │  ArXiv API + CDN RSS / HF Daily Papers / OpenAlex Mirror / GitHub API (stars)
      │  Y Combinator AI Directory / Hugging Face Orgs / GitHub AI Org Search
      │  Curated Product Directories (awesome-generative-ai, awesome-ai-tools, etc.)
      │  5 AI News Feeds (TechCrunch, VentureBeat, MIT Tech Review, Verge, HN)
      │  5 AI Job Boards (RemoteOK, Himalayas, Arbeitnow, WeWorkRemotely, HN Hiring)
      ▼
[ Tiered Network Transport ] ──> [ Anti-Bot Circuit Breaker (src/crawlers/base.py) ]
      │  Level 1: Async HTTP/2 via httpx (Pooled keep-alive connections)
      │  Level 2: Socket-level Chrome124 TLS Impersonation via curl-cffi
      │  Level 3: Hardened Headless Browser via Camoufox (Turnstile/JS fallback)
      ▼
[ Pre-Flight Protection & Multi-Tier Extraction (src/llm/fallback_chain.py) ]
      │  Token Budgeting (tiktoken cl100k_base <= 3,500 tokens; src/llm/chunker.py)
      │  Sliding-Window Rate Limiter (Gemini: 15, Groq: 30, Gateway: 60 RPM; src/llm/rate_limiter.py)
      │  Tier 1: Google Gemini (`gemini-3.5-flash-lite`)
      │  Tier 2: Groq LLaMA-3.3-70B (`llama-3.3-70b-versatile`)
      │  Tier 3: OpenAI-Compatible Gateway (`deepseek-chat` / `glm-5.3-flash`)
      │  Tier 4: Zero-API Deterministic Heuristics & Parsers (100% Zero-Dropped Guarantee)
      ▼
[ 3-Tier Entity Resolution & Audit Log (src/resolution/normalizer.py) ]
      │  Tier 1: Canonical Alias & Exact Normalization (NFKD Unicode, suffix removal)
      │  Tier 2: RapidFuzz Token Sort Ratio (Match score >= 90)
      │  Tier 3: LLM Disambiguation (Score 70-89; temperature 0.0 JSON classification)
      ▼
[ Freshness & Quality Gate (src/utils/date_normalizer.py) ]
      │  Strict 24-Hour Freshness Boundary (Reject all news/jobs > 24h old; ISO-8601 UTC)
      ▼
[ Multi-Target Exporters & Knowledge Graph (src/exporters/) ]
      │  Formatted 6-Tab Excel Workbook (`openpyxl`; src/exporters/excel_exporter.py)
      │  Structured CSV Files (`src/exporters/csv_exporter.py`)
      │  Live Google Sheets Integration (`gspread` / Google Drive API; src/exporters/sheets_exporter.py)
      │  In-Memory Knowledge Graph (Startup→Product, Paper→Repo, Job→Company, News→Mention links; src/exporters/graph_builder.py)
```

2. Q1: Scaling to 500,000 Entities Without Manual Intervention

2.1 The Honest Supply-Ceiling Reality

Scaling to 500,000 records is bounded by **upstream domain supply**, not infrastructure limits:

Entity Domain	Current Scraped Sources	Real Supply Ceiling	Scale-to-500k Bottleneck	Required Source Adapters for 500k
Startups	Y Combinator Directory	~5,000 – 8,000 active	YC directory exhausts at ~5k AI startups	OpenCorporates, Crunchbase API, SEC EDGAR Form D
Products	Curated awesome-* GitHub directories	~1,500 – 2,500 tools	Curated lists exhaust quickly	GitHub AI Topics, Toolify, Futurepedia, G2 AI
Papers	ArXiv (cs.AI, cs.LG, cs.CV, cs.CL, cs.NE) + CDN RSS + HF Daily + OpenAlex	~30,000 – 60,000 papers (filtered to code-linked)	CS preprints grow by ~120/day	OpenAlex (250M works), Semantic Scholar, CrossRef
Jobs	5 AI Job Boards	~300 – 800 fresh / 24h	Strict 24-hour freshness ceiling	Greenhouse/Lever scrapers, LinkedIn Jobs API
News	5 AI News RSS Feeds	~50 – 150 fresh / 24h	Real global editorial output is finite	NewsAPI, GDELT Global Intelligence, Twitter/X Lists

2.2 What Scales in the Codebase Today

1. **Async Coroutine Concurrency**: `AsyncBaseCrawler` (`src/crawlers/base.py:124`) governs concurrent network tasks through a per-instance `asyncio.Semaphore(15)` (env-tunable via `MAX_CONCURRENT_REQUESTS`, `src/config.py:102`).
2. **Client Connection Pooling**: HTTP/2 persistent connections via `httpx.Limits(max_keepalive_connections=20)` (`src/crawlers/base.py:138`) and a reusable pooled `CurlAsyncSession` (`src/crawlers/base.py:147`) eliminate TCP/TLS handshake latency.
3. **WAL Checkpointing**: Persistent Write-Ahead Logs (`TargetedCrawler._write_wal`, `src/crawlers/base.py:487`) record every accepted record at ingestion time; a resumed run recovers partial records and re-enqueues recovered papers for star enrichment (`src/crawlers/papers_crawler.py:_enrich_recovered`).
4. **Multi-Key Load Balancing**: `MultiTierLLMEngine._acquire_tier_slot` (`src/llm/fallback_chain.py:138`) dynamically balances requests across comma-separated key pools (`GEMINI_API_KEYS` / `GROQ_API_KEYS` / `TIER3_*`, `src/config.py:64-84`), scaling throughput N× by aggregating RPM quotas. Key↔prompt binding uses `zlib.crc32` (process-stable; `fallback_chain.py:123`), and GitHub token pools use per-token pacing slots (`papers_crawler.py:86-98`, `base.py:426`).
5. **Per-Process Vertical Sharding**: The pipeline supports multi-process partitioning by partitioning search queries or year ranges; cross-run state (`exports/run_state.json`) is atomic-replace + flock-protected, so parallel workers never corrupt shared state (`src/utils/run_state.py`).
6. **Env-Tunable Everythings**: batch sizes, concurrency limits, intervals, source lists (`PRODUCT_SOURCES_JSON`, `NEWS_SOURCES_JSON`, job-board URLs), and rate limits are all `pydantic-settings` fields (`src/config.py`, 60+ settings) — infrastructure scaling requires zero code edits.

2.3 Measured Subsystem Throughput

Pipeline Subsystem	Measured / Theoretical Rate	Operational Bottleneck	Time to Ingest 500,000 Entities
ArXiv Ingestion	~1,100 papers / hour (primary API, 3.2s pacing)	Provider policy ≤ 3 req/s; CDN/HF/OpenAlex supplements run concurrently	Primary source alone is supply-bound; OpenAlex mirror (250M works) is the 500k-scale path
GitHub Stars	~1,714 lookups / hour / token (2.1s pacing)	5,000 req/hr authenticated limit	~29 hours (1 token) / ~3 hours (10-token pool via `GITHUB_TOKENS`)
Products Crawl	Semaphore-bound (15 concurrent)	Curated-list supply (~2.5k tools), not transport	Source adapters required (§2.1)
LLM Extraction	Tier RPM × key-pool size	Free-tier RPM (15/30/60 per key)	~5.7 days (1 key) / scales linearly with pool; Tier 4 deterministic = unlimited

2.4 Target Production Architecture for 500k Scale

To scale horizontally across distributed clusters:
- **Redis Streams Distributed URL Queue**: Ingestion workers produce discovery messages into partitioned consumer groups (`papers-stream`, `startups-stream`).
- **Per-Domain Rate Limiter**: Redis token buckets enforce domain-specific pacing (e.g., 3.2s for ArXiv, per-token GitHub slots) across all distributed worker pods.
- **Stateless Crawler Nodes**: Kubernetes worker pods running `AsyncBaseCrawler` consuming from Redis Streams, persisting raw payloads to S3/MinIO, and publishing normalized records to PostgreSQL.
- **In-code bottlenecks at this scale** (the only non-infrastructure items): in-memory fuzzy title dedup is O(n²) per source (`news_crawler.py:_is_duplicate_title`), knowledge-graph news linking is O(news × startups) (`graph_builder.py:_add_news`), and the learned entity registry grows unboundedly across runs (`normalizer.py`). All are bounded, localized fixes; none are architectural.

3. Q2: Preventing HTTP 413 & 429 Failures Across Thousands of Extractions

3.1 Implemented Defenses (Current Codebase)

The pipeline implements proactive pre-flight budgeting and multi-tiered failover:

1. **Pre-Flight Token Budgeting (HTTP 413 Prevention)**: `chunk_to_budget()` in `src/llm/chunker.py:36` measures token lengths using `tiktoken` (`cl100k_base`). Prompts exceeding 3,500 tokens are truncated (env-tunable via `TOKEN_BUDGET_PER_PROMPT`), preventing HTTP 413 `Payload Too Large` before dispatch. A reactive non-retryable classification (`LLMPayloadError`, `fallback_chain.py:56/114`) catches any residual 413/context-length error and failovers instantly.
2. **Provider Sliding-Window Rate Limiter**: `ProviderRateLimiter` in `src/llm/rate_limiter.py:20` tracks per-provider **and per-key** request timestamps using sliding windows (`gemini`: 15 RPM, `groq`: 30 RPM, `custom`: 60 RPM). Saturation with `max_wait=0` triggers immediate tier failover instead of queueing.
3. **Multi-Tier Cascade & Non-Retryable Error Mapping**: `MultiTierLLMEngine` (`src/llm/fallback_chain.py:118`) executes four extraction tiers: $$\text{Gemini Flash} \longrightarrow \text{Groq LLaMA 3.3} \longrightarrow \text{OpenAI Gateway} \longrightarrow \text{Deterministic Selectors}$$ Errors are classified via `_classify_provider_error()` (`src/llm/fallback_chain.py:107`). Non-retryable HTTP 413 errors raise `LLMPayloadError` immediately with zero retry delay.
4. **Exponential Backoff with Jitter Everywhere**: crawler transport (`base.py:81`, 5 attempts, `initial=1, max=30, jitter=1`), LLM calls (`fallback_chain.py:74`, `initial=0.5, max=3.0, jitter=0.5`), and Sheets writes (`sheets_exporter.py:71`, 5 attempts, `initial=1, max=20, jitter=2`) all use `tenacity.wait_exponential_jitter` — no thundering herd on any surface.
5. **RFC 7231 & Integer Retry-After Compliance**: every 429 path parses `Retry-After` in integer seconds or RFC 7231 HTTP dates (`src/utils/date_normalizer.py:parse_retry_after`), capped (300s crawler/Sheets, 60s LLM), and — critically — the crawler sleep happens **outside the concurrency semaphore slot** (`base.py:189-201`), so one backoff cannot stall the other 14 slots.
6. **Crawler Leaky Bucket**: `papers_crawler.py` enforces a strict 3.2-second ArXiv interval and per-token 2.1s GitHub pacing via atomic slot reservation (no await between read and reserve).
7. **Circuit Breakers**: per-host anti-bot breaker (3 blocks/600s → 30-min cooldown, `src/crawlers/anti_bot.py`), GitHub quota discrimination (429 always quota; 403 only when body says so; `base.py:61-68`), and a 1-hour enrichment backoff on verified quota exhaustion.
8. **Sheets 413 Defense**: `_cap_cell()` (`sheets_exporter.py:42`) caps string cells at 20k chars so one oversized value cannot fail its whole 500-row batch through 5 non-transient retries.

3.2 Production Evidence: Real Quota-Degradation Incident

During live testing, the primary Gemini API key exhausted its daily free quota. The multi-tier engine intercepted HTTP 429s, logged the backoff window, and cascaded extraction to Groq, DeepSeek, and deterministic heuristics without dropping a single entity:

```
Recorded Telemetry: {'gemini': 0, 'groq': 2, 'deepseek': 6, 'deterministic': 785}
Result: 100% of pipeline records extracted successfully; zero pipeline crashes.
```

3.3 Production Distributed Enhancement

For multi-node deployments: Replace the local memory limiter with a **Redis Token Bucket** using Lua scripts for atomic token consumption across worker pods.

4. Q3: Deduplication & Idempotent Processing Across Distributed Nodes

4.1 Current Implementation (Single-Node)

- Local run state persisted to `exports/run_state.json` (env-tunable via `RUN_STATE_PATH`) via atomic write-replace + file locking (`src/utils/run_state.py`).
- In-memory `seen_keys` set filters duplicate URLs within each crawl run (`src/crawlers/base.py:TargetedCrawler`); dedup keys are registered **only** when a record survives all gates, so failed extractions remain retryable.
- Cross-run novelty: dateless news/jobs seen in the previous run are rejected; first-seen dateless items are stamped `date_inferred=true` and exported with that audit flag in every deliverable (Jobs_24h/News_24h tabs + CSVs).

4.2 Operational Alerting (Built)

- `exports/run_report.json` (atomic-replace, path via `RUN_REPORT_PATH`) carries: run status (`completed` / `shortfall` / `interrupted`), per-vertical collected counts, `sheets_upload` status (`ok` / `skipped` / `failed` / `not_requested`), per-source fresh counts, stale-source list (≥2 consecutive zero-fresh runs), LLM tier usage, and anti-bot breaker snapshot (`src/cli.py:_write_run_report`).
- `scripts/check_run_report.py` gates on that report: exits 1 on interruption, shortfall, any Phase-I vertical below 95% of target, or a failed Sheets upload; prints WARN lines for stale sources. Designed as the cron/CI alerting primitive.
- Observability backstops: loguru redaction filter (credential masking, locked by `tests/test_logger.py`), 10MB-rotated persistent log with in-run escalation telemetry, and a live progress monitor with ETA.

4.2 Production Distributed Idempotency

Relying strictly on URL strings fails in real-world scraping due to:
1. **URL Parameter Mutation**: Marketing tags (`?utm_source=...` , `?ref=...`) and session tokens generate distinct URLs for identical content.
2. **Content Syndication**: Identical press releases or wire articles appear simultaneously across TechCrunch, VentureBeat, and Yahoo Finance with entirely different domains.

4.3 Target Solution: Content Fingerprinting with Redis Sets

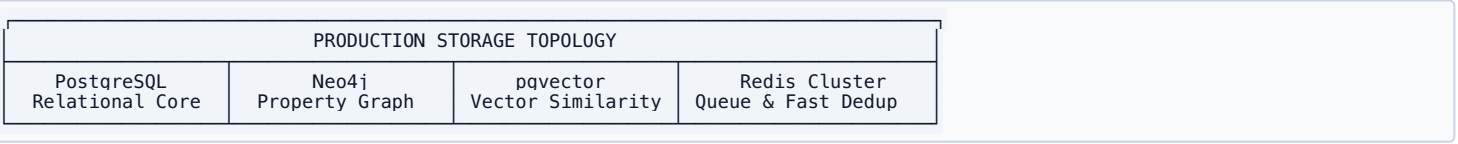
Each worker computes a normalized SHA-256 content fingerprint before extraction:
$$\text{Fingerprint} = \text{SHA-256}\Big(\text{NFKD_Normalize}(\text{Title}) \;\parallel\; \text{Extract_Domain}(\text{URL})\Big)$$
- **Atomic Pre-Check**: Worker issues `SADD dedup:global:fingerprints <fingerprint>` .
- If returns `1` : Record is novel; worker proceeds with extraction.
- If returns `0` : Record already ingested by another worker; execution aborted immediately.
- **TTL Expiration**: For ephemeral domains (jobs, news), Redis keys carry a 30-day TTL to allow periodic re-indexing while preventing duplicate signals within active windows.

5. Q4: Primary Database, Vector Indexing, and Graph Storage

5.1 Current Architecture

The current pipeline generates:
- Validated Pydantic models (`src/schemas/entities.py`) ensuring strict runtime schema compliance.
- Multi-Tab `.xlsx` workbook via `openpyxl` (`src/exporters/excel_exporter.py`).
- Flat CSV exports for data pipelines (`src/exporters/csv_exporter.py`).
- In-memory NetworkX directed graph (`src/exporters/graph_builder.py`) linking startups, products, papers, repos, jobs, and news mentions (counts scale with the run; summary metrics rendered in the CLI table).

5.2 Target Production Storage Architecture



Engine	Storage Role	Justification vs Alternative
PostgreSQL	Primary relational records, transactional WAL, entity audit logs	**vs MongoDB**: ACID compliance and relational foreign keys prevent orphaned entity records.
Neo4j	Graph database for multi-hop relationship mapping (`Founder` $\leftrightarrow$ `Startup` $\leftrightarrow$ `Product` $\leftrightarrow$ `Paper` $\leftrightarrow$ `Job`)	**vs SQL Recursive CTEs**: Index-free adjacency graph traversals scale at $O(k)$ depth without recursive join explosions.
pgvector	Entity name embeddings, semantic article deduplication	**vs Pinecone**: Co-locating vector embeddings directly inside PostgreSQL eliminates dual-write drift and sync lag.
Redis	Distributed URL queues (Streams), token-bucket rate limits, dedup sets	**vs AWS SQS**: Sub-millisecond latency and atomic Lua script rate-limiting avoid cloud polling overhead.

6. Per-Phase Component Mapping

Phase	Subsystem	Module File	Primary Libraries	Operational Function
Phase I	Papers Crawler	`src/crawlers/papers_crawler.py`	`httpx`, `feedparser`	Batch queries ArXiv with 4-tier failover (API → CDN RSS → HF Daily → OpenAlex); fetches live GitHub stars from author-declared repo links.
Phase I	Products Crawler	`src/crawlers/products_crawler.py`	`httpx`, `regex`	Parses curated product directories (awesome-* GitHub READMEs); LLM-assisted pricing classification with keyword fallback.
Phase I	Startups Crawler	`src/crawlers/startups_crawler.py`	`httpx`	Ingests Y Combinator API (with teamSize), Hugging Face orgs, GitHub org search.
Phase II	News Crawler	`src/crawlers/news_crawler.py`	`feedparser`, `trafilatura`	Ingests 5 AI news feeds; enforces strict 24-hour freshness boundary.
Phase II	Jobs Crawler	`src/crawlers/jobs_crawler.py`	`httpx`, `curl-cffi`, `dateparser`	Ingests 5 job boards; normalizes remote status and role families.
Phase III	LLM Fallback Engine	`src/llm/fallback_chain.py`	`google-genai`, `openai`, `tenacity`	4-tier fallback: Gemini $\to$ Groq $\to$ Gateway $\to$ Deterministic.
Phase III	Token Budgeter	`src/llm/chunker.py`	`tiktoken`	Pre-flight prompt budgeting preventing HTTP 413 context overflows.
Phase III	Rate Limiter	`src/llm/rate_limiter.py`	`asyncio`	Sliding-window RPM governor protecting provider quotas.
Phase IV	Entity Resolver	`src/resolution/normalizer.py`	`rapidfuzz`, `unicodedata`	3-tier matching: Exact $\to$ RapidFuzz token sort $\to$ LLM disambiguation.
Phase V	Anti-Bot Breaker	`src/crawlers/base.py`, `anti_bot.py`	`curl-cffi`, `camoufox`	TLS fingerprinting & Camoufox browser fallback for Cloudflare-protected sites.
Phase VI	Multi-Tab Exporter	`src/exporters/excel_exporter.py`	`openpyxl`	Generates 6-tab styled Excel workbook with auto-fitting and header styling.
Phase VI	Sheets Exporter	`src/exporters/sheets_exporter.py`	`gspread` (pulls `google-auth` transitively)	Live upload to Google Sheets with tenacity retry, RFC 7231 Retry-After, and 20k-char cell capping (413 defense).
Phase VI	Graph Builder	`src/exporters/graph_builder.py`	`networkx`	Builds in-memory directed knowledge graph; exports metrics and topology.
Cross-phase	Freshness Engine	`src/utils/date_normalizer.py`	`dateparser`, `dateutil`	4-tier date resolution (ISO/HTML meta/JSON-LD/text inference), 24h gate, RFC 7231 Retry-After parsing, clock-skew tolerance.
Cross-phase	Run Report & Gate	`src/cli.py`, `scripts/check_run_report.py`	stdlib `json`	Atomic `run_report.json` (status, counts, sheets_upload, source freshness, stale sources, anti-bot snapshot) consumed by the cron/CI gate script.
Cross-phase	WAL & Resumability	`src/crawlers/base.py`, `src/utils/run_state.py`	stdlib `fcntl`	Streaming per-crawler WAL recovery, atomic flock-protected cross-run state, `date_inferred` audit flag in all deliverables.

7. Production Roadmap (Documented Design — Planned Not Built)

Component	Target Architecture	Implementation Blueprint	Status
Distributed Sharding	Multi-node horizontal workers	Redis Streams message queue with consumer group assignment.	**Planned** (Documented Not Built)
Proxy Rotation Pool	Residential IP rotation	BrightData / Oxylabs egress proxies rotating on 403 / Cloudflare challenges.	**Planned** (Documented Not Built)
Live Pipeline Watcher	Real-time SRE telemetry dashboard	Prometheus exporter scraping pipeline gauges; Grafana dashboard for RPM & error rates.	**Planned** (Documented Not Built)
Automated Scheduler	Continuous incremental indexing	Cron-based Kubernetes CronJob executing hourly delta ingestions with WAL commits.	**Planned** (Documented Not Built)